

W niniejszej instrukcji zostały wypisane checkpointy, przez które należy przejść. Przy każdym znajduje się symbol ☐ , który zostanie sprawdzony przy oddawaniu laboratorium lub na konsultacjach.

SKRÓCONA CHECKLISTA

- ☐ Aplikacja powinna możliwie wiernie odwzorowywać dostarczony prototyp lub makietę
- ☐ Każdy ekran z prototypu musi być dostępny przez React Router
- ☐ Widoki powinny być rozdzielone na osobne komponenty stron w folderze pages.
- ☐ Powtarzające się elementy UI powinny być wydzielone do osobnych komponentów.
- ☐ Aplikacja powinna być ostylowana i czytelna wizualnie.
- ☐ Aplikacja musi zawierać działające logowanie użytkownika zrealizowane przy użyciu Firebase Authentication.
- ☐ Projekt powinien mieć zintegrowane narzędzie do analizy zachowań użytkowników — Hotjar
- ☐ Projekt powinien mieć zintegrowane Google Analytics
- ☐ Deploy aplikacji
- ☐ Dokumentacja readme.md dostarczona z kodem projektu, zawierająca screeny aplikacji, screeny użycia z google analytics, screeny aplikacji w hotjar

ocena:

- Odwzorowanie prototypu — 2 pkt
- Routing wszystkich ekranów — 2 pkt
- Podział na pages — 1 pkt
- Komponenty reużywalne — 2 pkt
- CSS / stylowanie — 1 pkt
- Logowanie — 2 pkt
- Hotjar — 1 pkt
- Google Analytics — 1 pkt
- Deploy aplikacji — 1pkt
- dokumentacja z screenami aplikacji, wynikami z hotjar i google analytics — 3 pkt

SZCZEGÓŁY

□ Aplikacja powinna możliwie wiernie odwzorowywać dostarczony prototyp lub makietę.

Co sprawdzamy

- układ sekcji
- kolejność elementów
- nagłówki, przyciski, formularze, karty, menu
- spacing, wyrównanie i podstawową responsywność
- zgodność widoków z projektem

Każdy ekran z prototypu powinien być zaimplementowany w aplikacji. Nie trzeba kopiować projektu co do piksela, ale interfejs ma być rozpoznawalny i spójny z makietą. Jeżeli w prototypie na Figma są ekrany:

- Login
- Dashboard
- Dodawanie wpisów
- Wyświetlanie raportów
- Profil

to wszystkie te widoki muszą istnieć w aplikacji.

□ Każdy ekran z prototypu musi być dostępny przez React Router

Co sprawdzamy

- czy routing działa
- czy każdy ekran ma przypisaną trasę
- czy można przejść pomiędzy ekranami bez przeładowania strony
- czy istnieje fallback dla nieistniejących ścieżek (404)

Jeśli projekt ma 5 ekranów, to każdy z nich powinien mieć osobną trasę, np. /, /login, /dashboard, /profile, /settings.

przykład:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import HomePage from "./pages/HomePage";
import LoginPage from "./pages/LoginPage";
import DashboardPage from "./pages/DashboardPage";
import ProfilePage from "./pages/ProfilePage";
import NotFoundPage from "./pages/NotFoundPage";
```

```
function App() {  
  return (  
    <BrowserRouter>  
      <Routes>  
        <Route path="/" element={<HomePage />} />  
        <Route path="/login" element={<LoginPage />} />  
        <Route path="/dashboard" element={<DashboardPage />} />  
        <Route path="/profile" element={<ProfilePage />} />  
        <Route path="*" element={<NotFoundPage />} />  
      </Routes>  
    </BrowserRouter>  
  );  
}  
  
export default App;
```

Dodatkowy plus

- chronione trasy po zalogowaniu
- zagnieżdżony routing
- layout wspólny dla kilku stron

☐ **Widoki powinny być rozdzielone na osobne komponenty stron w folderze pages.**

Folder **pages** powinien zawierać główne widoki aplikacji. Strona to pełny ekran lub główny widok związany z routingiem. Przykładowa struktura projektu:

```
src/  
  components/  
  pages/  
    HomePage.jsx  
    LoginPage.jsx  
    DashboardPage.jsx  
    ProfilePage.jsx  
  App.jsx  
  main.jsx
```

☐ **Powtarzające się elementy UI powinny być wydzielone do osobnych komponentów.**

Co sprawdzamy

- czy wspólne elementy są wydzielone
- czy komponenty są używane wielokrotnie

- czy komponenty przyjmują props

Jeśli ten sam element pojawia się kilka razy, np. *przycisk*, *karta produktu*, *pole formularza*, *navbar*, *modal* — powinien zostać wydzielony do **components**.

Przykład kodu dla komponentu przycisku:

```
function Button({ children, onClick, type = "button" }) {  
  return (  
    <button type={type} onClick={onClick} className="btn">  
      {children}  
    </button>  
  );  
}  
  
export default Button;
```

☐ Aplikacja powinna być ostylewana i czytelna wizualnie.

Co sprawdzamy

- czy style są faktycznie użyte
- czy interfejs jest spójny
- czy zastosowano jedną metodę stylowania konsekwentnie
- czy layout nie jest „surowy” HTML bez formatowania

☐ Aplikacja zawiera działające logowanie użytkownika zrealizowane np. przy użyciu Firebase Authentication.

należy:

1. utworzyć projekt w Firebase,
2. dodać aplikację webową do projektu,
3. zainstalować pakiet firebase,
4. skonfigurować initializeApp,
5. skonfigurować getAuth(),
6. włączyć logowanie Email/Password w Firebase Console,
7. przygotować formularz logowania,
8. obsłużyć zalogowanie i wylogowanie,
9. zabezpieczyć wybrane trasy.

☐ Projekt powinien mieć zintegrowane narzędzie do analizy zachowań użytkowników — Hotjar.

Hotjar najlepiej inicjalizować na poziomie głównego komponentu aplikacji, np. w App.jsx albo main.jsx.

instrukcja:

```
npm install @hotjar/browser
```

przykład inicjalizacji:

```
import { useEffect } from "react";
import Hotjar from "@hotjar/browser";

function App() {
  useEffect(() => {
    const siteId = 123456;
    const hotjarVersion = 6;

    Hotjar.init(siteId, hotjarVersion);
  }, []);

  return <div>Moja aplikacja</div>;
}

export default App;
```

☐ Projekt powinien mieć zintegrowane Google Analytics

W aplikacji React z routingiem trzeba pamiętać, że przejścia między podstronami nie przeładowują strony, więc page view trzeba śledzić przy zmianie lokalizacji.

instalacja:

```
npm install react-ga4
```

inicjalizacja w [App.js](#):

```
import { useEffect } from "react";
import ReactGA from "react-ga4";

function App() {
  useEffect(() => {
    ReactGA.initialize("G-XXXXXXXXXX");
  }, []);

  return <div>Moja aplikacja</div>;
}

export default App;
```

dodanie listenera w components/AnalyticsListener:

```
import { useEffect } from "react";
import { useLocation } from "react-router-dom";
import ReactGA from "react-ga4";

function AnalyticsListener() {
  const location = useLocation();

  useEffect(() => {
    ReactGA.send({
      hitType: "pageview",
      page: location.pathname + location.search,
    });
  }, [location]);

  return null;
}

export default AnalyticsListener;
```

użycie w [App.js](#):

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import AnalyticsListener from "../components/AnalyticsListener";

function App() {
  return (
    <BrowserRouter>
      <AnalyticsListener />

      <Routes>
        {/* route'y */}
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

☐ Deploy aplikacji

Prosty deploy aplikacji w oparciu o darmowe rozwiązania z opcją "one click deploy" np.
<https://docs.railway.com/guides/react>

☐ Dokumentacja [readme.md](#) dostarczona z kodem projektu, zawierająca screeny

aplikacji, screeny użycia z google analytics, screeny aplikacji w hotjar